

LLM 预训练之 LM Head

Output Projection / Softmax / Vocab Parallel / Cross-Entropy • 深度研究手册

从 hidden state 到 vocabulary probability: 为什么最后一层同时是表达瓶颈、显存热点和梯度接口

核心视角: LM Head = vocabulary classifier + probability interface + gradient return path

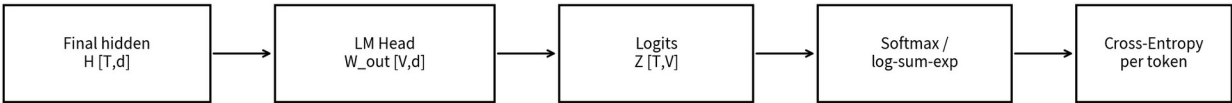
Research cut-off: 2026-08-13 | 01-B-Agent

执行摘要：LM Head 真正决定了什么

Decoder Transformer 最后一层得到 hidden state $h_t \in R^d$ 后，必须把它变成 vocabulary 上的概率分布。最常见做法是一张输出矩阵 $W_{out} \in R^{V \times d}$: $z_t = W_{out} h_t$, 再对 z_t 做 softmax / cross-entropy。公式看似简单，但当 V 达到 128K、256K 甚至更大时，这一层同时牵涉参数量、显存、通信、梯度几何与训练稳定性。

问题	核心机制	工程后果
Hidden → Vocabulary	d 维 hidden 投影为 V 维 logits	LM Head FLOPs/参数与 V 线性增长
Tied vs Untied	E_{in} 与 W_{out} 是否共享	节省 $V \times d$ 参数，但合并两类梯度角色
Softmax Bottleneck	线性投影 + softmax 的表达受低维 hidden 限制	输出分布族可能受 rank 限制
Large-Vocab Memory	训练时 logits 形状约为 $T \times V$	loss/head 可能成为显存热点
Vocab Parallel	沿 V 切 W_{out} 与 logits	把参数/激活分散到多个 TP ranks
Fused / Cut CE	避免或减少完整 logits 落 HBM	显著降低训练 loss 阶段内存
Gradient Return Path	vocab-space loss gradient 需经 W_{out}^T 回到 d 维	LM Head 不只是输出层，也是 backbone 的反馈接口

LM Head 是 Transformer hidden space 到 vocabulary probability space 的最后接口



数学上只是 $Z = H W_{out}^T$ ；系统上却可能生成 $T \times V$ 的巨型中间张量

图 1 标准 autoregressive LM 的输出链路。T 表示本次参与 loss 的 token positions。

最重要的 Know-why LM Head 不能只按“最后一层参数”评估。它决定了模型如何把高维上下文表征映射成离散 token 竞争，同时也决定 cross-entropy 的大部分 vocabulary 级梯度如何返回 backbone。

1. 数学本体：从 hidden state 到 logits

对一个 token position t , 标准输出层为 $z_t = W_{out} h_t + b$, 其中 $h_t \in R^d$, $W_{out} \in R^{V \times d}$, $z_t \in R^V$ 。现代 decoder LLM 常省略 bias。预测概率 $p_i = \exp(z_i) / \sum_j \exp(z_j)$, 真实 token y 的 loss 为 $L_t = -\log p_y$ 。

对象	形状	含义	主要成本
hidden H	$[T, d]$	Transformer 最终表示	来自 backbone
W_{out}	$[V, d]$	每个 vocabulary item 的 classifier vector	参数约 $V \times d$
logits Z	$[T, V]$	所有 token 对所有词表项的未归一化分数	训练内存约 $T \times V \times \text{bytes}$
softmax probs	$[T, V]$ 或流式计算	概率分布	通常无需长期保存全部概率
CE loss	$[T]$	每个位置的监督信号	最终 reduction 后得到 batch loss

参数成本非常直接：若 $V=128K$ 、 $d=8192$ ，则单独一张 untied LM Head 约有 1.05B 参数；BF16 权重约 2.1GB。若再有独立输入 embedding，词表接口两端合计就可能超过 2B 参数。

2. Weight Tying：为什么曾经成为经典默认

Press & Wolf 2017 将 topmost output matrix 明确视为一种 output embedding，并提出把 input embedding 与 output embedding 绑定；论文报告在多种语言模型上降低 perplexity，并显著减少参数量。Inan et al. 也从 loss framework 推导出 input/output tying。[1][2]

Weight tying 的本质不是“共享参数”这么简单，而是共享两个不同训练角色

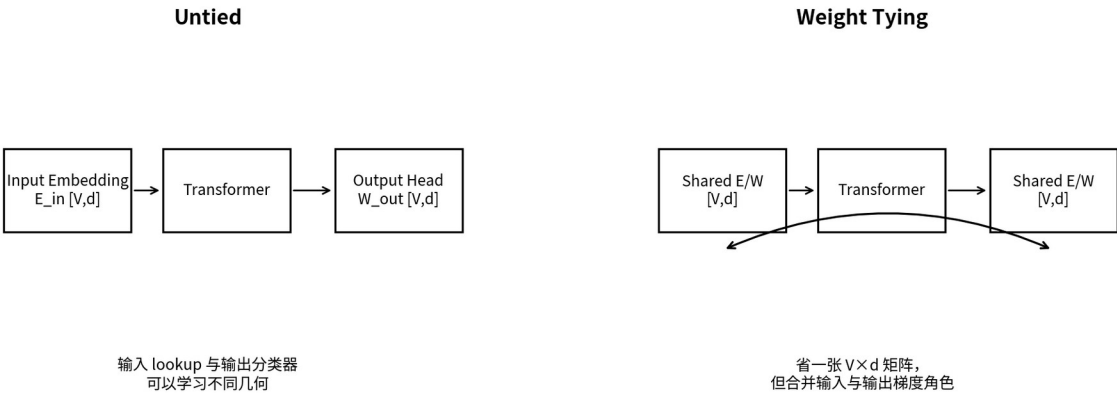


图 2 Tied 与 untied 的真正差异：不仅少一张矩阵，还决定输入 lookup 与输出分类器是否必须共享同一几何。

维度	Tied	Untied
参数量	省去一张 V×d 矩阵	两张独立 V×d
输入角色	token identity lookup	token identity lookup
输出角色	同一矩阵同时作为 classifier	独立 classifier geometry
梯度	输入与输出信号合并	两类更新解耦
适用倾向	参数预算紧、历史 LM 常见	现代大 decoder 模型常见之一
风险	强行假设输入/输出几何可共享	额外参数与内存

2024 ICML 的分析进一步指出，tying 隐含了输入语义空间与输出上下文分布空间之间的结构假设，因此它不是普适的“免费午餐”。现代 Llama 配置默认 tie_word_embeddings=False；Gemma 2 则选择 tied，并配合 final logit soft-capping，说明两条路线都仍在生产模型中存在。[3][9][10]

Know-how 做 tied/untied ablation 时必须固定总参数或至少报告参数差异。否则 untied 模型多出来的 V×d 参数本身就可能影响结果，不能把全部差异归因于“共享是否更合理”。

3. Softmax Bottleneck：输出分布的表达力为什么受 d 限制

Yang et al. 2018 将语言建模写成矩阵分解问题，指出线性 hidden-to-logits + softmax 的可表达分布存在 rank 限制，即所谓 softmax bottleneck；Mixture of Softmaxes 用多个 softmax components 提升有效 rank。[4]

直觉上，如果所有上下文 c 的 log-probability vector 都必须通过一个 d 维 hidden 再乘同一 W_{out} 产生，那么 vocabulary 维度虽然是 V，但可自由变化的方向首先受 d 维表示限制。当 V≫d 时，这种低维结构可能无法覆盖自然语言真实条件分布的全部复杂性。

路线	如何缓解瓶颈	优势	代价/现状
Increase d	提高 hidden rank 上限	直接、通用	全模型成本大幅上升
Mixture of Softmaxes	多个 softmax experts 混合	经典表达力提升方法	输出计算更复杂
Nonlinear output transform	在线性 logits 后加可学习非线性	可提升有效 rank	主流 LLM 中并非常见默认
Untied / richer head	放松输入输出共享、增加 head 自由度	工程简单	仍保留线性 d→V 主瓶颈

注意 Softmax bottleneck 是表达能力分析，不等于“大模型一定被 LM Head 卡死”。现代 LLM 的 d 已远大于早期 RNN LM，且最终能力受数据、backbone、objective 等共同影响；应通过受控 pretraining ablation 验证实际收益。

4. Large Vocabulary：LM Head 为什么会突然变成显存热点

训练时如果显式物化 $Z = H W_{out}^T$ ，logits 张量大小与 $T \times V$ 成正比。序列越长、micro-batch 越大、vocabulary 越大，最终 loss 前的临时激活可能非常可观。

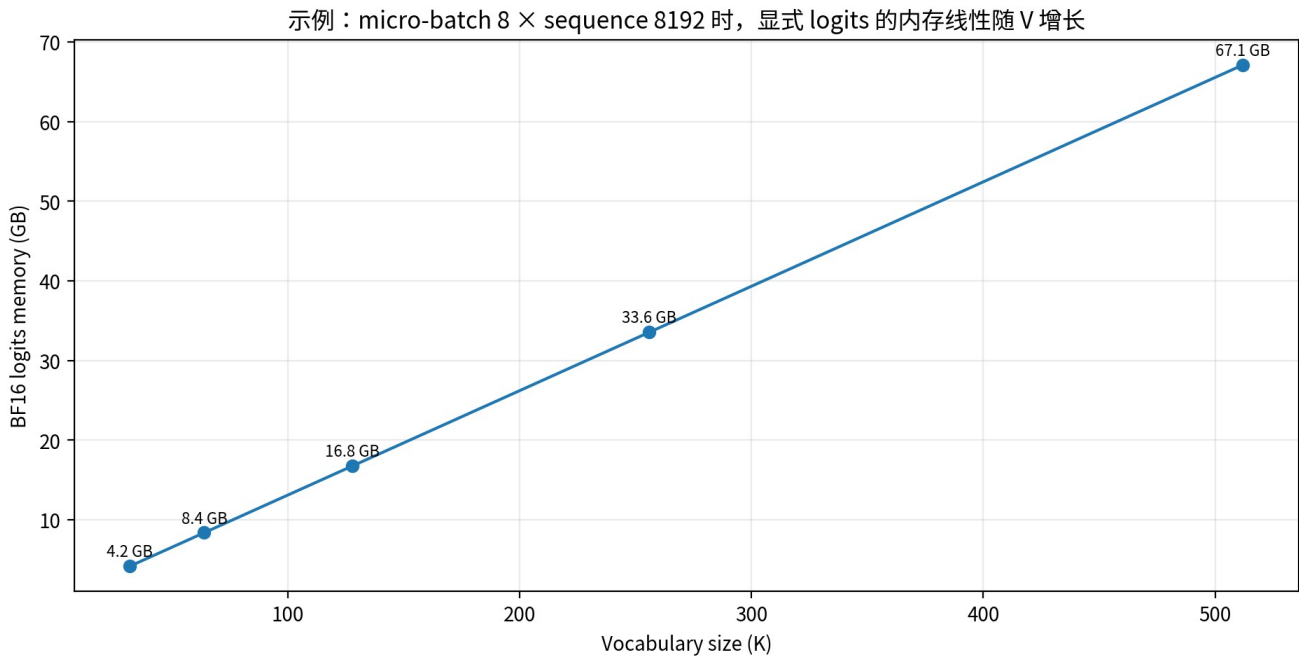


图 3 仅作数量级示例：micro-batch=8、sequence=8192、BF16 logits。这里未计算 backward buffers、softmax workspace 与并行策略。

Cut Cross-Entropy (CCE) 2024 针对这一问题提出不把所有 logits 写入 global memory，而是在 kernel 内流式计算 target logit 与 log-sum-exp。论文报告在 Gemma 2 2B 案例中，将 loss computation memory 从约 24GB 降到约 1MB，并把 classifier-head 总训练内存从约 28GB 降到约 1GB，同时保持收敛。[7]

成本项	传统 full logits	Memory-efficient CE 思路
W_out GEMM	必须	仍需计算必要 dot products
T×V logits 写 HBM	通常是	避免/最小化
log-sum-exp	对完整 logits reduction	tile / streaming reduction
target logit	从完整矩阵 gather	直接计算/局部获取
backward	依赖完整或重算 softmax 信息	融合 kernel / 重算 / 稀疏忽略极小贡献

5. Full Softmax vs Adaptive / Sampled / Hierarchical

在现代 GPU full-softmax 大规模化之前，large-vocabulary LM 已长期研究 adaptive softmax、hierarchical softmax、target sampling、NCE 等近似方法。Grave et al. 的 adaptive softmax 利用词频长尾，将高频词与低频词分层分簇，以降低期望计算量。Chen et al. 2015 系统比较了 softmax、hierarchical softmax、target sampling、NCE 与 self-normalization。[5][6]

方法	核心思想	是否 exact full CE	适合场景
Full Softmax	所有 V 类都参与归一化	是	现代 dense LLM 默认；质量语义最直接

方法	核心思想	是否 exact full CE	适合场景
Adaptive Softmax	高频直出、低频分簇	近似/结构化	超大词表、历史 LM / 特定任务
Hierarchical Softmax	沿树路径预测	否	极大类别空间
Sampled Softmax / NCE	只采一部分 negatives	否	减少输出计算；需处理 estimator bias/variance
CCE / fused exact CE	仍保持 full-softmax objective，但不物化完整 logits	是	现代大词表 LLM 系统优化

现代趋势 大模型预训练更倾向保留 exact full-softmax objective，同时从 kernel、memory traffic 与 tensor parallel 角度优化，而不是优先改变学习目标。原因是近似 softmax 会同时改变优化统计与工程复杂度。

6. Vocab Parallel：怎样把 $V \times d$ Head 分布到多 GPU

Megatron Core 提供 VocabParallelEmbedding 与 vocab_parallel_cross_entropy：把 vocabulary 维度切到 tensor-parallel ranks，每个 rank 只持有 V/TP 的 output weights 与 local logits，再通过 collective operations 计算 global max、sum-exp 与目标 token logit。这样无需每张 GPU 都物化完整 V 。[11][12]

Vocab Parallel：沿 vocabulary 维度切 LM Head，而不是把完整 V 复制到每张 GPU

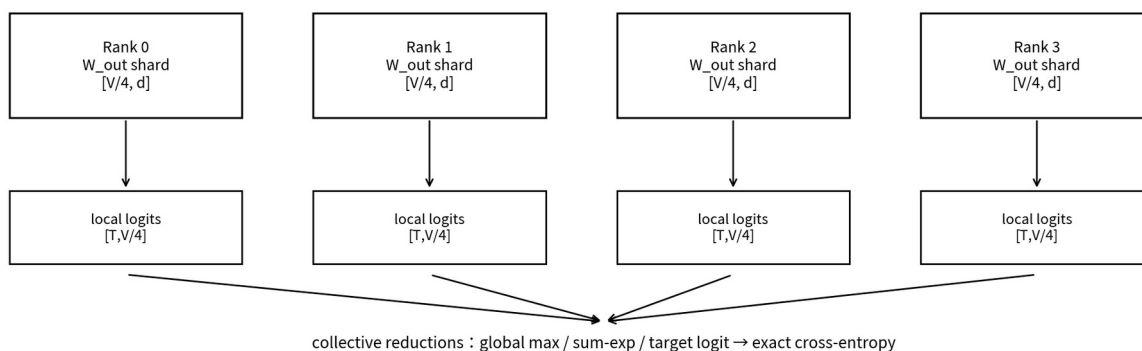


图 4 Vocab Parallel Cross-Entropy 的核心：保持 exact cross-entropy，但把 vocabulary 维度分片。

操作	本地计算	需要通信
logits max	每个 rank 求 local max	all-reduce max 得 global max
exp / sum	每个 rank 对本地 vocab shard 计算	all-reduce sum 得 global denominator
target logit	只有包含 target id 的 rank 有值	all-reduce sum / masked reduction
gradient	每 rank 得到 local vocab gradient	按 TP 设计继续回传到 hidden

这也是为什么 LM Head 的并行策略与 Tokenizer 的 vocabulary size 直接耦合： V 越大，vocab parallel 的收益越明显，但通信 reduction 也更关键。

7. Logit Scale / Soft-capping：输出分数过大为什么是稳定性问题

Cross-entropy 对 logits 的绝对平移不敏感，但对相对尺度非常敏感。若 logit norm 持续增大，softmax 会越来越尖锐；错误类别梯度分布、数值范围与优化动态都会变化。

Gemma 2 的公开配置包含 final_logit_softcapping=30，并对最终 logits 使用 tanh soft-capping；这说明生产模型已经把“输出 logit scale”作为显式架构/稳定性控制参数，而不仅仅交给优化器自行调节。[10]

监控量	异常表现	可能动作
logit RMS / max	持续单调上升、极端 outlier	检查 LR、final norm、softcap、初始化
entropy	过早塌缩到极低	检查 logit scale 与数据重复/过拟合
target margin	极端增大但 eval 不升	可能出现过度置信
top-k mass	过早集中	检查 temperature-like scale dynamics
CE grad norm at head	突然 spike	结合 backbone grad 与 mixed precision 排查

8. LM Head 是 Gradient Return Path，不只是出接口

对一个 token，loss 对 hidden state 的梯度为 $\partial L / \partial h = W_{\text{out}}^T (p - \text{onehot}(y))$ 。也就是说，所有 vocabulary 级别的“谁应该升、谁应该降”的反馈，最终都必须经过 W_{out}^T 压回 d 维 residual space，然后才进入整个 Transformer backbone。

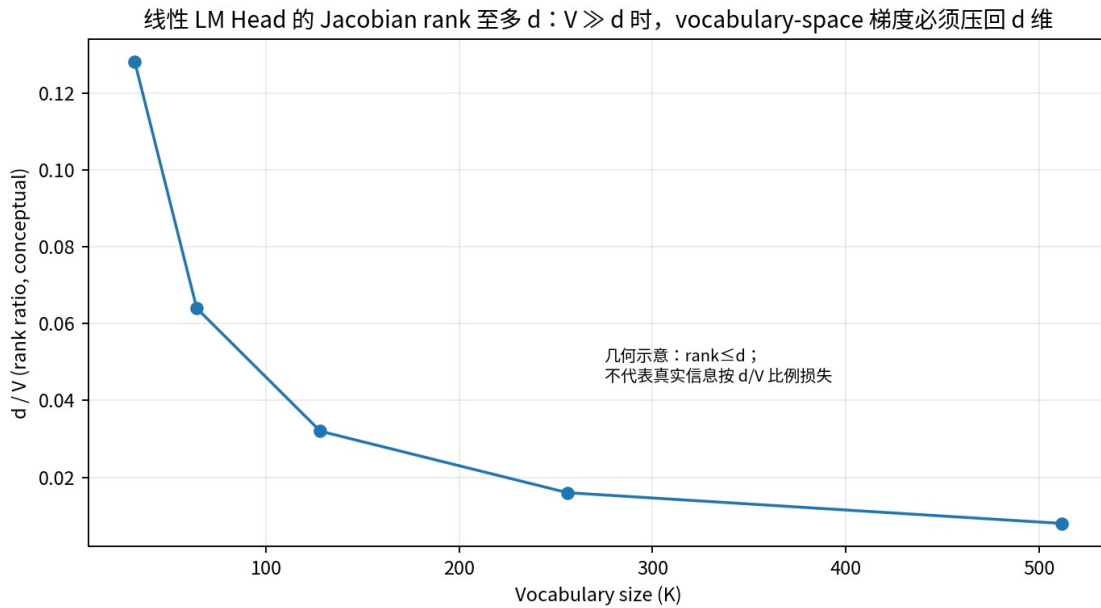


图5 线性 head 的几何事实： W_{out}^T 的 rank 至多为 d 。图示只表达 $V \gg d$ 时存在压缩，不代表真实梯度信息按 d/V 比例损失。

2026 年 Godey & Artzi 的预印本把这一点进一步形式化为 LM-head gradient bottleneck，并报告其测量中 95-99% 的 vocabulary-space gradient norm 可在投影回 hidden space 时被抑制，同时通过受控预训练实验观察到学习动态受影响。该结论很新，尚应视为前沿证据而非已经成为社区共识的定论。[13]

重要边界 “softmax bottleneck” 讨论输出分布表达 rank；“gradient bottleneck” 讨论反向传播反馈经过低秩线性层后的压缩。两者相关但不是同一个问题。

9. Rare Tokens：为什么输出层反而比输入 Embedding 更新更密集

输入 embedding 的某一 row 只有对应 token 真正在输入中出现时才收到直接 lookup gradient；而 full-softmax LM Head 的每个 row 在每个预测位置都通过 p_i 参与分类梯度（数值大小可能极小）。因此 low-frequency token 的 output classifier vector 仍然可以频繁获得“作为负类”的更新。

Token 频率	Input embedding direct update	Output head update	潜在现象
高频 token	很频繁	很频繁	输入/输出都充分训练
低频 token	稀疏	每个位置都有负类项，正类很少	输出向量可能主要由 negative pressure 塑造

Token 频率	Input embedding direct update	Output head update	潜在现象
新增 token	扩词后初始直接更新少	立即参与所有 softmax negatives	初始化与 warmup 特别重要

因此词表扩展时只讨论新 embedding row 的初始化是不够的；若 untied，还必须同时初始化/训练新 output rows，并观察 rare-token logit bias、norm 与 positive-hit learning speed。

10. Tied vs Untied：现代模型为什么仍然没有统一答案

Llama 的 Transformers 配置默认 tie_word_embeddings=False；Gemma 2 配置则为 True。两者说明是否 tying 不是“新旧架构”的单一分界，而是参数预算、词表大小、训练 recipe 与模型设计共同决定的选择。[9][10]

条件	更偏向 Tied	更偏向 Untied
参数预算	希望节省 $V \times d$ 参数	参数预算允许
Vocab 很大	节省参数收益显著	但输出分类几何独立的价值也变大
词表扩展	一次扩 E/W 即可	输入/输出两端分别初始化
多语言/代码混合	共享可形成统一 token 几何	输入语义与输出竞争关系可能需要解耦
研究目标	参数效率、历史可比性	输出层优化、梯度/几何分析

实践上应把 tying 当成 architecture hyperparameter，用固定 token budget 的 proxy pretraining 做受控实验，而不是默认套用“共享一定更好”或“现代模型一定 untied”。

11. LM Head Observability：训练时至少监控什么

类别	指标	为什么重要
Scale	hidden RMS、logit RMS/max、W_out row norm	判断输出层是否异常放大
Distribution	entropy、top-1/top-k mass、target margin	判断 softmax 是否过早尖锐/塌缩
Gradient	head grad norm、hidden grad norm、row-wise grad by frequency bucket	定位 head/backbone 梯度失衡
Rare token	positive hit count、row norm、rare-token loss	检测长尾词是否学到
System	LM-head GEMM time、CE time、HBM peak、TP comm time	定位输出层是否成吞吐瓶颈
Numerics	Inf/NaN、softmax max-subtract、BF16/FP8 overflow stats	防数值故障

Know-how 把“head time / total step time”和“CE peak memory / total peak memory”单独记录。大 vocabulary 下，优化 backbone kernel 可能几乎不改变吞吐，因为真正的瓶颈已移动到 output + loss。

12. 严格 Ablation：怎样知道 LM Head 改动真的有效

实验变量	必须固定/报告	评价维度
Tied vs Untied	总参数差异、V、d、token budget	PPL、下游、rare token、吞吐
Vocab size	raw-text budget 与 tokenizer 效率	loss/byte 或 bits/byte + downstream
Soft-capping	LR、final norm、precision	稳定性、entropy、PPL、loss spikes
CCE / fused CE	同一 exact objective	peak memory、tokens/s、收敛一致性
Vocab Parallel degree	DP/TP/PP 其他配置一致	通信、MFU、HBM、step time
Alternative head	参数/FLOPs matching	表达收益 + optimization + system cost

尤其不要用不同 tokenizer 的 token-level perplexity 直接比较不同 vocabulary 方案。若 tokenizer 变化，应同时报告基于原始文本长度归一化的指标，如 bits-per-byte / nats-per-byte，或在统一下游任务上评估。

13. Production Blueprint：现代大词表 LM Head 的推荐设计流程

阶段	建议动作	退出条件
1. Vocab/Tokenizer joint design	确定 V、token efficiency、special tokens	原始文本压缩与语言公平性达标
2. Head topology	tied/untied、bias、final norm、softcap	proxy model ablation 明确
3. Parallel plan	Vocab Parallel / TP degree	单 rank 参数和 logits 可容纳
4. Loss kernel	fused CE / CCE 类 exact kernel	peak memory 与吞吐达标
5. Stability	监控 logit scale / entropy / head grad	无持续 scale drift / spike
6. Long-tail audit	frequency bucket eval	rare/new tokens 不出现系统性退化
7. Scale transfer	小模型→目标规模重复验证	收益在更大 d/V、长 horizon 下仍存在

Production 默认起点 如果没有强证据支持改变 objective：优先保留 exact full-softmax cross-entropy；把优化重点放在 vocab parallel、fused/streaming CE、logit-scale observability 与 tied/untied proxy ablation。

14. Know-Why / Know-How 总结

Know-why	Know-how
LM Head 是 vocabulary classifier，不只是 linear layer	用 V×d 参数、T×V activation、CE kernel 三张账同时评估
Weight tying 合并两种不同优化角色	做参数匹配 ablation，并看输入/输出 row geometry 与梯度
V>d 带来输出表达与反馈压缩问题	区分 softmax bottleneck 与 gradient bottleneck
大词表使 loss 变成显存热点	优先用 vocab parallel + exact fused/streaming CE
logit scale 会影响 softmax 几何与稳定性	监控 RMS/max/entropy/margin，必要时验证 soft-capping
rare token 的输入/输出训练统计不同	按频率 bucket 监控 row norm、positive hit 与 loss
Head 优化不能脱离 tokenizer	Vocab size、token efficiency、raw-text budget 联合实验

参考资料与证据等级

优先列一手论文、同行评审论文与官方实现/文档。2026 年新预印本单独标记为“前沿证据”，不与成熟结论等量处理。

[1] Press & Wolf (2017), Using the Output Embedding to Improve Language Models. <https://aclanthology.org/E17-2025/> - 同行评审；weight tying 经典证据

[2] Inan, Khosravi & Socher, Tying Word Vectors and Word Classifiers. <https://arxiv.org/abs/1611.01462> - 经典理论/实验

[3] Bertolotti & Cazzola (ICML 2024), By Tying Embeddings You Are Assuming the Distributional Hypothesis. <https://openreview.net/forum?id=yyYMAprcAR> - 同行评审；tying 假设分析

[4] Yang et al. (ICLR 2018), Breaking the Softmax Bottleneck. <https://openreview.net/forum?id=HkwZSG-CZ> - 同行评审；softmax bottleneck

[5] Grave et al., Efficient softmax approximation for GPUs / Adaptive Softmax. <https://arxiv.org/abs/1609.04309> - 经典 large-vocab output method

[6] Chen, Grangier & Auli, Strategies for Training Large Vocabulary Neural Language Models. <https://arxiv.org/abs/1512.04906> - large-vocab 方法系统比较

[7] Wijmans et al. (2024), Cut Your Losses in Large-Vocabulary Language Models. <https://arxiv.org/abs/2411.09009> - CCE；memory-efficient exact CE

[8] Meta, The Llama 3 Herd of Models. <https://arxiv.org/abs/2407.21783> - 现代大词表模型技术报告

[9] Hugging Face Transformers - LlamaConfig. https://huggingface.co/docs/transformers/model_doc/llama - 官方实现文档；tie_word_embeddings=False

[10] Hugging Face Transformers - Gemma2Config. https://huggingface.co/docs/transformers/model_doc/gemma2 - 官方实现文档；tied + final logit softcapping

[11] NVIDIA Megatron Core - VocabParallelCrossEntropy. https://docs.nvidia.com/megatron-core/developer-guide/latest/apidocs/core/core.tensor_parallel.cross_entropy.html - 官方系统文档

[12] NVIDIA Megatron Core - Tensor Parallel Layers / VocabParallelEmbedding. https://docs.nvidia.com/megatron-core/developer-guide/latest/apidocs/core/core.tensor_parallel.layers.html - 官方系统文档

[13] Godey & Artzi (2026), Lost in Backpropagation: The LM Head is a Gradient Bottleneck. <https://arxiv.org/abs/2603.10145> - 前沿预印本；需后续复现与评审

[14] Gemma Team (2024), Gemma 2: Improving Open Language Models at a Practical Size. <https://arxiv.org/abs/2408.00118> - 现代模型技术报告